Real Estate

360° Virtual Tour Platform

Full Implementation Guide

System Architecture • Seller Capture Flow
Buyer Viewer Experience • Technology Stack
Database Schema • API Design • MVP Roadmap

Prepared for internal development reference

1. Project Overview

This document provides a full implementation blueprint for a real estate platform that enables property sellers to capture and publish interactive 360° virtual tours, and allows buyers to explore properties immersively from any device — desktop, mobile, or VR headset.

The experience is inspired by automotive virtual showrooms (such as Kia's VR Showroom), adapted for real estate. The key difference: automotive uses 3D model rendering for exteriors and photographic spheres for interiors — in real estate, all rooms use photographic 360° spheres stitched from captures taken on-site.

Core Value Proposition

For sellers: list a property with a professional virtual tour in under 30 minutes, no technical expertise needed.

For buyers: explore every room of a property from anywhere, at any time, before visiting in person.

For the platform: increase listing quality, reduce time-on-market, and differentiate from flat-photo listings.

1.1 What Type of 360° Experience Is This?

This is NOT a pre-recorded video tour. It is an interactive, navigable photo sphere experience — the same technology used in Google Street View and Matterport, powered by equirectangular photography stitched into full spherical panoramas.

Feature	Description
Exterior view	360° equirectangular photo of the property facade and surroundings
Room-by-room interior	One or more spherical panoramas per room, linked via hotspots
Navigation hotspots	Clickable arrows to move between rooms (e.g. living room → kitchen)
Feature callouts	Clickable markers highlighting key property features
Floor plan minimap	Shows buyer's current location within the property layout
VR mode	WebXR stereo-split view for use with a VR headset or Google Cardboard

2. Full System Architecture

The platform has three distinct layers: the Seller Capture Pipeline, the Backend Processing Platform, and the Buyer Viewer. These three layers communicate via APIs and a shared CDN for media delivery.

2.1 Architecture at a Glance

Layer	What it does	Who interacts
Seller mobile/web app	Captures 360° images, uploads, tags rooms, sets hotspots	Property owner / estate agent
Backend API server	Receives uploads, triggers processing, stores metadata	Internal — seller app & buyer viewer
Media processing service	Stitches frames, compresses, generates tiles	Automated pipeline
Database	Stores listings, rooms, hotspot positions, metadata	Internal
CDN / tile server	Serves optimised panorama tiles to buyers globally	Buyer viewer
Buyer web viewer	Renders the 360° tour, handles navigation, hotspots, VR	Property buyer / browser

2.2 Data Flow

The following sequence describes how a listing goes from an on-site visit to a live virtual tour:

1. Seller visits the property and opens the capture app on their smartphone or 360° camera.
2. They capture one panorama per room (living room, bedroom, kitchen, bathroom, etc.) plus the exterior.
3. The app uploads the raw equirectangular images (or raw frames for stitching) to the backend.
4. The backend processing pipeline stitches frames (if needed), compresses, and tiles the panoramas.
5. Seller completes the listing: labels rooms, adds navigation hotspots between rooms, adds feature callouts.
6. Listing goes live. The buyer viewer loads the tour on demand from the CDN.

3. Seller Capture Pipeline

This is the most critical UX challenge: sellers are not technical users. The capture experience must be guided, foolproof, and fast. A seller should be able to complete a full property tour capture in under 20 minutes.

3.1 Three Capture Tiers

Offer three tiers to accommodate different seller types:

Tier	Hardware	Best for	Output quality
Tier 1 (Recommended)	Consumer 360° camera (Ricoh Theta, Insta360 X4, GoPro Max)	Most sellers — agent or self-listing	High — full equirectangular in one shot
Tier 2 (DIY)	Smartphone + your guided capture app	Budget sellers, no extra hardware needed	Medium — stitched from burst frames
Tier 3 (Premium)	Matterport Pro2, FARO, or Leica scanner	Luxury listings, agencies	Very high — full 3D mesh + dollhouse view

3.2 Seller App — Step-by-Step Capture Flow

The seller app (iOS + Android, built in React Native or Flutter) guides the seller through each room with on-screen instructions:

Step 1: Property Setup

Seller creates a new listing: address, property type, number of rooms.
App auto-generates a room list (Living Room, Bedroom 1, Kitchen, etc.).
Seller can add custom rooms or rename existing ones.

Step 2: On-site Room Capture

For Tier 1 (360° camera): seller pairs the camera via Bluetooth/Wi-Fi. App shows a live preview. Seller stands in the centre of the room and taps capture.
For Tier 2 (smartphone): app enters guided panorama mode — an on-screen guide tells the seller to slowly rotate 360°. App captures burst frames automatically.
App confirms a green tick when the panorama is complete. Seller moves to the next room.

Step 3: Upload & Processing

App uploads images over Wi-Fi (or queues for later upload if on cellular).
Backend stitches frames and returns a preview thumbnail within 60 seconds.
Seller sees a processing indicator per room.

Step 4: Hotspot Placement

Seller opens each room preview in the app's hotspot editor.

They drag-and-drop navigation arrows (to link rooms) and feature pins (to annotate features).

Example: in the kitchen, seller places a pin on the appliances with the text 'Bosch integrated oven, 2023'.

The floor plan minimap is uploaded at this stage (photo or drawn outline).

Step 5: Publish

Seller reviews the completed tour via an in-app preview.

One tap publishes the tour. It goes live on the listing within seconds.

4. Backend Architecture & Processing Pipeline

4.1 Technology Stack — Backend

Component	Recommended technology	Purpose
API server	Node.js + Express or FastAPI (Python)	REST API for seller app and buyer viewer
Media processing	Krpano, Pano2VR, or custom ffmpeg pipeline	Stitch frames, generate tiles, compress
Object storage	AWS S3 or Google Cloud Storage	Store raw uploads and processed panoramas
CDN	CloudFront (AWS) or Cloudflare	Fast global delivery of panorama tiles
Database	PostgreSQL	Listings, rooms, hotspots, user accounts
Queue	BullMQ (Redis) or AWS SQS	Async processing of uploaded images
Auth	Firebase Auth or Auth0	Seller and buyer authentication

4.2 Media Processing Pipeline

When a seller uploads raw images or frames, the following automated pipeline runs:

- Step 1 — Ingest: File lands in S3. An S3 event triggers a Lambda function or a queue worker.
- Step 2 — Stitch (if raw frames): Hugin or Microsoft ICE (via CLI) stitches frames into a single equirectangular image. If the seller used a 360° camera, this step is skipped.
- Step 3 — Colour correct: Adjust exposure and white balance for consistency across rooms.
- Step 4 — Tile generation: Krpano slices the equirectangular into multi-resolution cube faces or tiles (like a map zoom system), so the viewer can load only the tiles currently visible — critical for performance.
- Step 5 — Preview generation: A small JPEG thumbnail is extracted for the room selector UI.
- Step 6 — Metadata update: Database is updated with tile paths, thumbnail URLs, and status = 'ready'.

4.3 API Design — Core Endpoints

```
POST   /api/listings                  Create a new property listing
GET    /api/listings/:id              Get listing with all rooms + tour data
POST   /api/listings/:id/rooms        Add a room to a listing
POST   /api/rooms/:id/upload          Upload raw panorama image(s) for a room
GET    /api/rooms/:id/status          Check processing status (pending/ready/error)
PUT    /api/rooms/:id/hotspots        Save hotspot positions for a room
POST   /api/listings/:id/publish      Make a listing's tour publicly visible
GET    /api/tour/:id                  Public tour data (rooms, hotspots, tiles) for
viewer
```

5. Database Schema

The following tables form the core data model for the virtual tour system:

5.1 listings table

```
id              UUID PRIMARY KEY
seller_id       UUID REFERENCES users(id)
address         TEXT NOT NULL
city            TEXT
price           NUMERIC
property_type   TEXT  -- apartment, house, villa
status          TEXT  -- draft, processing, live, archived
floor_plan_url  TEXT  -- uploaded floor plan image
created_at      TIMESTAMP
published_at    TIMESTAMP
```

5.2 rooms table

```
id              UUID PRIMARY KEY
listing_id      UUID REFERENCES listings(id)
name            TEXT  -- 'Living Room', 'Master Bedroom', etc.
order_index     INT   -- display order in room selector
tile_path       TEXT  -- CDN path to processed tiles
thumbnail_url   TEXT  -- small preview image
status          TEXT  -- uploading, processing, ready, error
initial_yaw     FLOAT -- starting camera direction (degrees)
initial_pitch   FLOAT -- starting camera pitch
```

5.3 hotspots table

```
id              UUID PRIMARY KEY
room_id         UUID REFERENCES rooms(id)
type            TEXT  -- 'navigation' | 'feature' | 'measurement'
target_room_id  UUID REFERENCES rooms(id)  -- for navigation type
yaw             FLOAT  -- horizontal angle in panorama (degrees)
pitch           FLOAT  -- vertical angle in panorama (degrees)
label           TEXT   -- display text
description     TEXT   -- popup content for feature hotspots
icon            TEXT   -- icon name
```

6. Buyer Viewer — Front-End Implementation

The buyer viewer is a web application embedded into property listing pages. It must work on desktop browsers, mobile, and optionally VR headsets. The viewer loads tour data from the API, then renders the panoramas using WebGL.

6.1 Technology Options for the Viewer

Library	Complexity	Control	Best for
Pannellum (open source)	Low	Medium	MVP — fast to implement, good feature set
Three.js (custom)	High	Full	Custom branding, full design control, advanced features
A-Frame	Medium	Medium-high	Built-in WebXR/VR support, declarative HTML syntax
Matterport Embed SDK	Very low	Low	Premium listings using Matterport hardware
Krpano (JS)	Medium	High	Industry standard, XML-driven, used by many real estate platforms

Recommendation for MVP

Start with Pannellum or A-Frame for speed of development.

Migrate to a custom Three.js viewer once product-market fit is established and you need full design control.

6.2 Viewer Features — Detailed Breakdown

Navigation Hotspots

Navigation hotspots are placed at the doorways and openings between rooms. When a buyer clicks one, the viewer transitions to the linked room's panorama. The transition should be animated (a smooth fade or a zoom-in-then-fade) to avoid disorientation.

```
// Pannellum hotspot config example
{
  type: 'scene',           // navigation type
  sceneId: 'bedroom-1',    // ID of the linked room
  pitch: -10,              // vertical angle in panorama
  yaw: 120,                // horizontal angle in panorama
  text: 'Master Bedroom', // tooltip text
  cssClass: 'nav-hotspot' // custom styling
}
```

Feature Callout Hotspots

Feature hotspots are placed on notable elements: appliances, materials, smart home devices. Clicking them opens an overlay card with the feature name, description, and optionally a photo. These are configured by the seller during the listing process.

Floor Plan Minimap

A small semi-transparent floor plan image is overlaid in the corner of the viewer. As the buyer navigates between rooms, a dot indicator moves to show their current position. This is implemented as an absolutely-positioned HTML element over the WebGL canvas, with position coordinates stored per room in the database.

Room Selector Bar

A scrollable horizontal bar at the bottom of the viewer shows thumbnail previews of all rooms. Clicking a thumbnail jumps directly to that room's panorama. On mobile, this collapses into a compact list to save screen space.

VR / Cardboard Mode

Using the WebXR Device API (or the A-Frame vr-mode-ui component), the viewer can enter a stereo split-screen mode compatible with Google Cardboard and basic VR headsets. For a full WebXR implementation with Three.js, the VRButton helper from three/examples/jsm/webxr/VRButton enables this with a few lines of code.

6.3 Viewer Component Structure

```
TourViewer (root component)
  ├── PanoramaCanvas       (Three.js / WebGL sphere renderer)
  ├── HotspotLayer         (SVG overlay for navigation + feature pins)
  ├── RoomSelectorBar      (bottom thumbnail strip)
  ├── FloorPlanMinimap     (corner overlay with position indicator)
  ├── FeatureCalloutPanel  (slide-in panel for feature details)
  ├── VRModeButton         (WebXR entry point)
  └── InquiryOverlay       (CTA button: Book a visit / Contact agent)
```

7. Full Technology Stack Summary

Layer	Technology	Rationale
Seller mobile app	React Native or Flutter	Single codebase for iOS + Android
Buyer web viewer	React + Three.js (or A-Frame for MVP)	Full control + WebXR support
API server	Node.js + Express or Python FastAPI	Fast development, strong ecosystem
Image processing	Krpano CLI + ffmpeg + Hugin	Industry-standard panorama tools
Object storage	AWS S3 (or GCS)	Scalable, cheap, reliable
CDN	AWS CloudFront or Cloudflare R2	Global low-latency tile delivery
Database	PostgreSQL + PostGIS (optional)	Relational + geospatial queries
Job queue	BullMQ (Redis-backed)	Async media processing pipeline
Auth	Firebase Auth or Clerk	Managed auth, low setup cost
Hosting	AWS ECS / Render / Railway	Containerised API + worker deployment
360° camera SDK	Ricoh Theta SDK / Insta360 Open SDK	Bluetooth camera pairing in seller app

8. MVP Development Roadmap

The following phased plan delivers a working virtual tour product in 3 months, then iterates toward a premium experience.

Phase 1 — Core Tour (Weeks 1–6)

- Seller web dashboard: create listing, upload pre-made equirectangular images (no guided capture yet)
- Backend: upload to S3, basic tile generation with Krpano CLI, status tracking
- Buyer viewer: Pannellum-based viewer embedded in listing page, basic room navigation
- Database: listings, rooms, hotspots schema
- Basic hotspot editor: seller places navigation arrows between rooms

Phase 2 — Seller Capture App (Weeks 7–10)

- React Native app with guided panorama capture mode (smartphone, no extra hardware)
- Insta360 / Ricoh Theta Bluetooth SDK integration for Tier 1 camera users
- In-app hotspot editor (drag-and-drop pins on room preview)
- Feature callout hotspots with text and photo

Phase 3 — Buyer Experience (Weeks 11–14)

- Migrate buyer viewer to custom Three.js for full design control
- Floor plan minimap with room position indicator
- VR / Google Cardboard mode via WebXR
- Room dimension annotations (seller enters measurements during listing)
- Inquiry / Book a visit CTA embedded in viewer

Phase 4 — Premium & Scale (Month 4+)

- Matterport integration for luxury listings (3D dollhouse view)
- Analytics: track which rooms buyers spend most time in
- Aerial exterior view via drone capture integration
- AI-powered automatic hotspot suggestions based on image content
- White-label SDK for estate agents to embed in their own websites

9. Key Architectural Decisions

The following decisions will have the most impact on development speed, cost, and product quality:

Decision	Option A	Option B	Recommendation
Capture pipeline	Build your own stitching + tiling	Use Matterport or Krpano as black box	Use Krpano for tiling (MVP), own the stitching later
Seller capture UX	Guide smartphone users through panorama capture	Require a 360° camera	Support both: smartphone for accessibility, 360 cam for quality
Viewer library	Pannellum (fast)	Custom Three.js (flexible)	Pannellum for MVP, migrate to Three.js at Phase 3
Hosting	Self-hosted on EC2/ECS	Fully managed (Render, Railway)	Managed hosting initially, migrate when cost demands it
VR support	WebXR (browser-native)	Native VR app	WebXR only — native app is significant extra investment

10. Security & Performance Considerations

10.1 Security

- All media uploads must be validated server-side: file type, file size, and image dimensions.
- Seller authentication required for all listing management endpoints (JWT or session token).
- Listing tour data can be public (no auth) or private (share link only) — seller configures this.
- Rate-limit upload endpoints to prevent abuse.
- Store Matterport or third-party API keys in environment variables, never in client code.

10.2 Performance

- Use multi-resolution tiling (like map zoom levels) so the viewer loads only the tiles currently in the viewport — essential for fast load on mobile.
- Serve tiles via CDN from the region nearest the buyer.
- Preload the next room's tiles in the background while the buyer is viewing the current room.
- Use WebP format for tiles where supported (30–50% smaller than JPEG with no perceptible quality loss).
- Target: first room renders in under 3 seconds on a 4G mobile connection.

11. Summary

Building a real estate 360° virtual tour platform is achievable in a focused 3-month sprint for the core experience. The key insight from Kia's VR Showroom is that the viewer experience is a hybrid: navigation between rooms uses photographic spheres (not 3D rendering), enriched with interactive hotspots for features and wayfinding.

The hardest problem is not the viewer — it is the seller capture flow. Making it fast, guided, and hardware-agnostic will determine adoption. Focus Phase 1 on nailing the buyer experience with manually-uploaded images, then build the seller capture app once the viewer is proven.

Next Steps

1. Define your target seller persona: self-listing homeowner, or professional estate agent?
2. Decide on minimum viable camera support: smartphone-only, or include 360° camera from Day 1?
3. Set up Krpano evaluation license and test the tile pipeline with sample 360° images.
4. Build the Pannellum-based buyer viewer prototype and test with a single room panorama.
5. Design the database schema and REST API spec before writing any front-end code.